INTERNSHIP PROJECT REPORT

CIFAKE DEEPFAKE IMAGE DETECTOR

Deep-Learning-Based REAL/FAKE Image Classification and Flask Web Deployment

Intern

Mimansha Goyal

Declaration

I hereby declare that this report presents the work carried out during my internship project entitled **CIFAKE Deepfake Image Detector**. The implementation, experiments, evaluation, and web deployment described in this report correspond to the work completed during the internship. Wherever external research or published material has been used, it has been appropriately acknowledged in the references.

Mimansha Goyal

Acknowledgement

I would like to express my sincere gratitude to everyone who supported me during my internship and helped me complete this project successfully. I also acknowledge the researchers and organizations that made the CIFAKE dataset, EfficientNet research, and related synthetic-media detection literature available to the research community. Their work provided the technical foundation for this project.

Abstract

The rapid development of generative artificial intelligence has made it increasingly difficult to distinguish authentic visual content from synthetically generated content using visual inspection alone. This project presents a practical image-classification system named **CIFAKE Deepfake Image Detector**. The system is designed as a binary classifier that predicts whether an input image belongs to the REAL or FAKE class.

The project uses the CIFAKE dataset, organized into REAL and FAKE image classes. An EfficientNetB0 convolutional neural network with ImageNet-pretrained weights is used as the principal feature-extraction and classification architecture. Images are resized to 128×128 pixels and processed in batches of 32. In the completed training run, the project used 100,000 images for the training source, with 90,000 images used for training and 10,000 for validation, while 20,000 images were available in the official test directory. The recorded completed run reported an accuracy of 88.68%, precision of 88.00%, recall of 89.57%, and loss of 0.2744 for the evaluation output captured during the run.

The trained model was saved as `best_model.keras` and integrated into a Flask web application. The web interface allows a user to upload an image, preview it, submit it for analysis, and receive a REAL/FAKE prediction with confidence. Testing was performed with images from the training and test REAL/FAKE folders as well as external images. The project demonstrates an end-to-end workflow from dataset preparation and deep-learning model training to local web deployment. The report also identifies the limitations of dataset-specific detection and proposes future improvements such as broader datasets, cross-generator evaluation, explainable AI, video analysis, and robustness testing.

Keywords: CIFAKE, synthetic image detection, deep learning, EfficientNetB0, image classification, Flask, transfer learning, REAL/FAKE classification.

Contents

Declaration i

Acknowledgement ii

Abstract iii

1 Introduction 1

1.1 Project Background 1

1.2 Problem Statement 1

1.3 Objectives 2

1.4 Scope of the Project 2

1.5 Expected Outcomes 3

2 System Overview 4

2.1 Existing System 4

2.2 Proposed System 4

2.3 System Modules 5

2.4 Overall Workflow 6

3 Literature Survey 7

3.1 Research Papers Reviewed 7

3.1.1 CIFAKE: Image Classification and Explainable Identification of AI-Generated Synthetic Images 7

3.1.2 EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks 7

3.1.3 Deep Learning for Deepfakes Creation and Detection: A Survey 7

3.1.4 A Review of Deep Learning-based Approaches for Deepfake Content Detection 8

3.1.5 NIST Synthetic Image Detection and Evaluation Work 8

3.2 Existing Solutions 8

3.3 Comparative Analysis 9
3.4 Research Gap 9

4 Technologies Used **10**
4.1 Programming Languages 10
4.2 Frameworks and Libraries 10
4.3 Development Tools 11
4.4 Project Configuration 11

5 Architecture and Design **12**
5.1 System Architecture Diagram 12
5.2 Workflow Explanation 12
5.3 Data Flow 13

6 Implementation **14**
6.1 Key Features Implemented 14
6.1.1 Dataset Loading 14
6.1.2 Image Preprocessing 14
6.1.3 Model Construction 14
6.1.4 Model Training 14
6.1.5 Model Metadata 15
6.1.6 Flask Backend 15
6.1.7 Frontend 15
6.1.8 Inference Logic 15
6.1.9 Project Directory Structure 16

7 Results and Discussion **20**
7.1 Output Analysis 25
7.2 Performance Evaluation 25
7.3 Testing Results 26
7.4 Discussion 26

8 Challenges and Learning Outcomes **28**
8.1 Challenges Faced 28

8.2 Solutions Implemented 28
8.3 Technical Skills Learned 29

9 Future Enhancements **30**
9.1 Proposed Improvements 30
9.2 Additional Features 30
9.3 Scalability Opportunities 30

10 Conclusion **32**
10.1 Summary of Work Completed 32
10.2 Achievements 32
10.3 Project Impact 33
10.4 Final Remark 33

List of Figures

5.1 Layered system architecture 12
5.2 Inference data flow 13

6.1 CIFAKE dataset structure and dataset distribution 16
6.2 EfficientNetB0 model training over five epochs 17
6.3 Project structure of the CIFAKE detection system 18
6.4 Flask backend running successfully in the VS Code terminal 19
6.5 Web interface of the CIFAKE Deepfake Image Detector 19

7.1 Training and validation accuracy of the EfficientNetB0 model 20
7.2 Training and validation loss of the EfficientNetB0 model 20
7.3 Final test-set performance and classification report 21
7.4 Confusion matrix of the CIFAKE classification model 21
7.5 Example prediction for a REAL image 22
7.6 Example prediction for a FAKE image 23
7.7 Grad-CAM visualization for model interpretability 23
7.8 Detection result for a FAKE image 24
7.9 Detection result for a REAL image 24

List of Tables

3.1 Comparison of approaches relevant to the project 9

4.1 Major technologies and their roles 10

7.1 Recorded model performance from the completed run 25

7.2 Functional testing of the deployed application 26

CHAPTER 1

Introduction

1.1 Project Background

Generative artificial intelligence has significantly improved the ability of machines to create realistic visual content. Modern image-generation systems can produce images that contain realistic textures, lighting, objects, environments, and human-like details. As a result, visual realism alone is no longer a sufficient basis for determining whether an image is authentic. Research on synthetic-media detection has therefore focused on identifying statistical, visual, and learned features that separate authentic and generated content. A NIST technical document describes passive synthetic-image detection as analysis that does not require additional information embedded at capture time and instead looks for traces or inconsistencies left by image-generation or manipulation processes [5].

The CIFAKE dataset was introduced specifically as a benchmark for binary classification between real images and AI-generated images. The original CIFAKE work constructed synthetic images corresponding to CIFAR-10 classes and studied CNN-based classification, including explainability through Grad-CAM [1]. This project builds a practical application around the same problem definition while replacing a basic CNN-oriented implementation with an EfficientNetB0-based classifier and integrating the resulting model into a Flask web application.

EfficientNet was proposed by Tan and Le as a family of convolutional neural networks that balances network depth, width, and input resolution using compound scaling [2]. EfficientNetB0 is used in this project because it provides a compact and transferable architecture suitable for image classification and local web inference.

1.2 Problem Statement

AI-generated and synthetic images can be difficult to distinguish from authentic images, especially when the generated content is visually convincing. Manual inspection is subjective, time-consuming, and dependent on the observer's experience. At the same time, an image detector trained on a particular dataset may not generalize equally well to images generated by unseen models or modified through compression, resizing,

filtering, or other post-processing. Current research and evaluation initiatives continue to emphasize robustness and generalization as important challenges in synthetic-media detection [7, 6].

The problem addressed in this project is therefore to develop an end-to-end deep-learning system that can learn the distinction between REAL and FAKE images from the CIFAKE dataset and expose the trained classifier through an easy-to-use web interface. The system should support image upload, preprocessing, model inference, and presentation of the predicted class and confidence.

1.3 Objectives

The main objectives of the project are:

1. To study the problem of distinguishing real images from AI-generated synthetic images.
2. To prepare and use the CIFAKE REAL/FAKE dataset for supervised binary classification.
3. To develop an EfficientNetB0-based image classifier using ImageNet-pretrained weights.
4. To train and evaluate the classifier using training, validation, and test data.
5. To save the trained model in a reusable format for inference.
6. To integrate the trained model with a Flask backend and a browser-based frontend.
7. To test the deployed system using REAL and FAKE images from the project dataset and additional external images.

1.4 Scope of the Project

The project focuses on image-level binary classification. Its primary input is a single image uploaded through a web interface, and its output is a predicted class, REAL or FAKE, together with a confidence value. The implementation is intended as an internship prototype and demonstrates the complete machine-learning lifecycle from data loading and training to deployment.

The scope does not include forensic authentication of image provenance, guaranteed detection of every type of manipulation, video deepfake detection, or attribution of

an image to a particular generator. The system should therefore be interpreted as a learned classifier rather than a universal authenticity verifier.

1.5 Expected Outcomes

The expected outcomes are:

- A trained EfficientNetB0-based REAL/FAKE classifier.
- A saved model file that can be loaded without retraining.
- A Flask web application for image upload and prediction.
- Quantitative evaluation using accuracy, precision, recall, and loss.
- A documented testing workflow and project report.
- A foundation for future work on robustness, explainability, and generalization.

CHAPTER 2

System Overview

2.1 Existing System

Synthetic-image detection can be implemented using several families of techniques. Traditional forensic approaches may inspect metadata, compression artifacts, pixel-level inconsistencies, frequency-domain patterns, or traces of manipulation. NIST's overview of synthetic-image detection distinguishes active methods, which can rely on information added at capture time such as signatures or watermarking, from passive methods that inspect the image itself for traces of processing or generation [5].

Deep-learning approaches instead learn discriminative features from labeled examples. Survey literature shows that CNNs and other deep-learning architectures have been widely explored for deepfake and synthetic-content detection [3, 4]. Evaluation platforms such as NIST OpenMFC further demonstrate that image manipulation and deepfake detection are evaluated using explicit test protocols and confidence-based metrics [8].

2.2 Proposed System

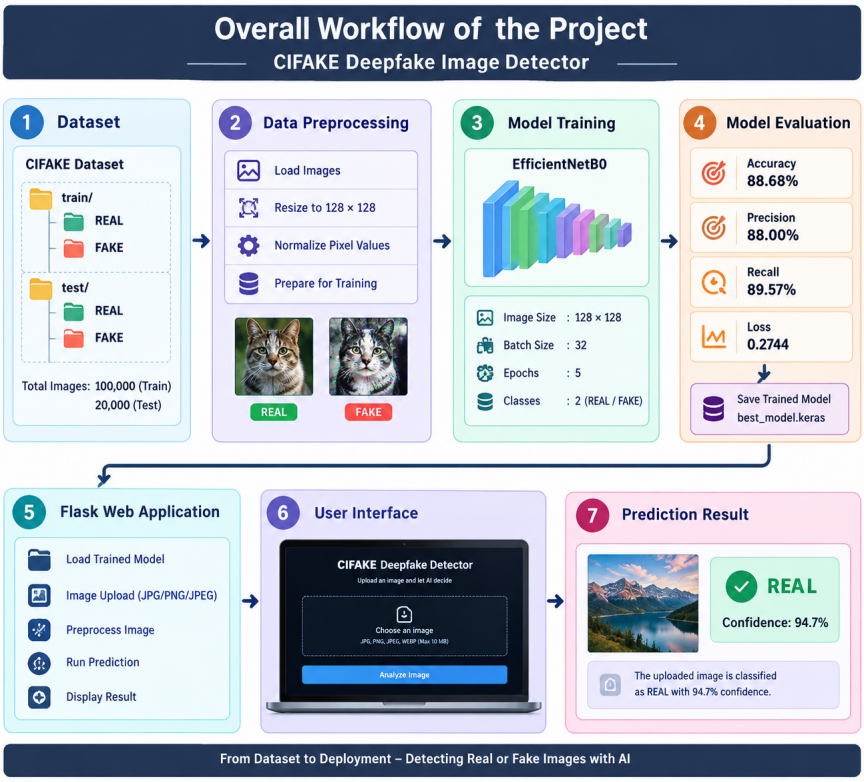
The proposed system uses a supervised binary image-classification pipeline. The dataset is organized into REAL and FAKE folders. During training, images are resized to 128×128 pixels and supplied to an EfficientNetB0-based classifier. The trained network produces a probability for the FAKE class. A threshold of 0.5 is used by the application to convert the model output into the final REAL or FAKE label.

The trained model is stored in the `models/` directory and loaded by the Flask backend. The frontend provides an upload control, preview, analysis button, and result panel. This separation allows the expensive training stage to be performed once while the web application performs only inference during normal use.

2.3 System Modules

1. **Dataset Module:** Loads the REAL and FAKE image directories and creates training, validation, and test datasets.
2. **Preprocessing Module:** Resizes images to 128×128 pixels and prepares batches for the neural network.
3. **Model Module:** Uses EfficientNetB0 with ImageNet-pretrained weights and a binary classification head.
4. **Training Module:** Trains the classifier and stores the best model as `best_model.keras`.
5. **Evaluation Module:** Computes classification metrics on the evaluation data used by the training script.
6. **Web Backend Module:** Flask receives uploaded files, performs preprocessing, loads the saved model, and returns the prediction.
7. **Frontend Module:** HTML and CSS provide the user interface for uploading images and displaying the result.
8. **Upload Storage Module:** Temporarily stores uploaded images under the web application's static upload directory so that the uploaded image can be displayed in the result page.

2.4 Overall Workflow



CHAPTER 3

Literature Survey

3.1 Research Papers Reviewed

The literature survey was conducted to understand the dataset, model architecture, deepfake-detection approaches, and current evaluation challenges.

3.1.1 CIFAKE: Image Classification and Explainable Identification of AI-Generated Synthetic Images

Bird and Lotfi introduced CIFAKE as a dataset and classification problem for distinguishing real images from AI-generated images. Their study generated synthetic counterparts of CIFAR-10 image classes and evaluated CNN architectures. The reported best model in their study achieved 92.98% accuracy, and Grad-CAM was used to investigate image regions contributing to classification [1]. This paper directly motivates the dataset and binary classification task used in the present project.

3.1.2 EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks

Tan and Le proposed EfficientNet by systematically balancing network depth, width, and resolution through compound scaling. Their experiments showed that the EfficientNet family can achieve strong image-classification performance with improved parameter and computation efficiency compared with several earlier architectures [2]. This work motivates the use of EfficientNetB0 as the main backbone in the present implementation.

3.1.3 Deep Learning for Deepfakes Creation and Detection: A Survey

Nguyen et al. reviewed deepfake generation and detection methods and described the growing role of deep learning in both creating and detecting synthetic visual content. The survey also highlights the difficulty of keeping detection systems effective

as generation methods improve [3]. This supports the need for continual evaluation rather than treating a detector as universally reliable.

3.1.4 A Review of Deep Learning-based Approaches for Deepfake Content Detection

Passos et al. reviewed deep-learning approaches for deepfake detection and discussed limitations and future directions. The review emphasizes that deepfake detection is an evolving problem in which generalization and robustness remain important concerns [4].

3.1.5 NIST Synthetic Image Detection and Evaluation Work

NIST documentation describes multiple approaches to synthetic-image detection and stresses the need for systematic evaluation. The 2025 NIST GenAI pilot evaluation plan explicitly frames image discriminators as systems that differentiate AI-generated images from human-generated images and proposes evaluation of strengths and weaknesses across modern generation systems [6]. NIST's more recent forensic work also highlights the gap that can occur between laboratory performance and real-world deployment, including changes caused by post-processing and newer generators [7].

3.2 Existing Solutions

Existing solutions can be grouped into four broad categories:

- **Metadata/provenance-based methods:** use signatures, provenance records, watermarks, or metadata when such information is available.
- **Forensic artifact methods:** inspect compression, frequency, pixel, or manipulation traces.
- **Deep-learning classifiers:** learn visual features directly from labeled real and synthetic images.
- **Hybrid and evaluation-oriented systems:** combine multiple evidence sources or evaluate detector robustness across generation and post-processing conditions.

3.3 Comparative Analysis

Table 3.1: Comparison of approaches relevant to the project

Approach	Main idea	Strength	Limitation
Metadata / provenance	Examine metadata, signatures, watermarking, or provenance information	Can provide strong evidence when trusted provenance is available	Evidence may be absent, removed, or unavailable
Forensic artifacts	Detect pixel, compression, frequency, or manipulation traces	Does not require a learned semantic classifier in every case	Artifacts can change with generation and post-processing
CNN-based CIFAKE study	Learn REAL/FAKE visual features from CIFAKE	Directly aligned with the dataset's binary task	Performance is tied to training distribution
EfficientNetB0 approach used here	Transfer learned image features to REAL/FAKE classification	Compact architecture and convenient deployment	External/generalization performance requires additional testing
Web-deployed detector	Integrates inference into an interactive Flask application	Makes the model directly usable by a non-technical user	Local prototype is not a forensic-grade production service

3.4 Research Gap

The literature establishes that deep-learning detectors can achieve high benchmark performance, but current research continues to identify generalization and operational robustness as open problems. NIST's recent forensic work explicitly studies robustness to newer generation methods and post-processing conditions [7]. The original CIFAKE study also focuses on a specific synthetic-image benchmark [1].

The gap addressed by this project is primarily an **implementation and deployment gap**: instead of stopping at model training, the project converts a CIFAKE-based classifier into an end-to-end application with a browser interface, saved model, inference backend, and repeatable demonstration workflow. The project also documents the limitation that benchmark-based classification does not automatically imply reliable performance on unseen generators.

CHAPTER 4

Technologies Used

4.1 Programming Languages

- **Python:** Used for dataset loading, preprocessing, model training, evaluation, and Flask backend development.
- **HTML:** Used to structure the web interface.
- **CSS:** Used for page layout, colors, cards, buttons, upload area, and prediction display.

4.2 Frameworks and Libraries

Table 4.1: Major technologies and their roles

Technology	Role in the project
TensorFlow / Keras	Deep-learning model construction, training, evaluation, and model loading
EfficientNetB0	Main convolutional backbone for image classification
Flask	Lightweight Python web framework used to serve the prediction application
Pillow	Image opening and conversion during inference
NumPy	Numerical array handling and probability processing
HTML5	Frontend structure
CSS3	Frontend styling and layout

EfficientNetB0 is available through Keras Applications and can be initialized with ImageNet-pretrained weights. TensorFlow documentation notes that EfficientNet includes its required input rescaling in the model implementation and expects pixel inputs in the 0–255 range for the standard application interface [10]. The project uses TensorFlow 2.20.0 in the Python virtual environment.

4.3 Development Tools

- **Visual Studio Code:** Primary development environment.
- **Python virtual environment (.venv):** Isolates project dependencies.
- **Command-line terminal:** Used for package installation, training, and starting the Flask server.
- **Web browser:** Used to interact with the deployed local application.

4.4 Project Configuration

The final environment used during development was based on Python 3.13.15 and TensorFlow 2.20.0. The project dependency file contains Flask 3.1.0, Pillow 11.3.0, and TensorFlow 2.20.0. TensorFlow's published build information lists TensorFlow 2.20.0 as tested with Python 3.9–3.13 for CPU builds, which is consistent with the working environment used for this project [9].

CHAPTER 5

Architecture and Design

5.1 System Architecture Diagram

The system architecture is divided into four logical layers: data, model, application backend, and user interface.

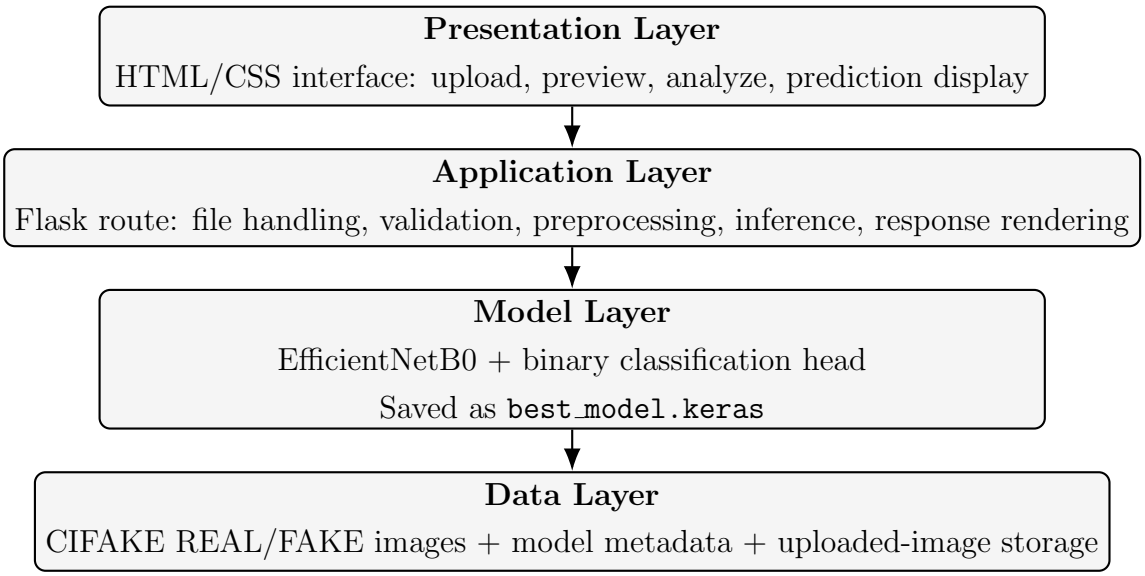


Figure 5.1: Layered system architecture

5.2 Workflow Explanation

The training workflow begins with the CIFAKE directory structure. The training directory contains REAL and FAKE classes, and the test directory contains the corresponding two classes. TensorFlow's directory-based dataset loader detects the two classes and produces batches. Ten percent of the training source is used for validation in the implemented training configuration.

The EfficientNetB0 model receives 128×128 RGB images. The pretrained convolutional backbone extracts visual features, and a binary classification layer produces a scalar output representing the model's confidence for the FAKE class. During inference, the same image-size requirement is applied before the image is passed to the saved model.

After training, the best model is stored under `models/best_model.keras`. The Flask application loads this file when the server starts. When the user submits an image, the backend reads the file, converts it to RGB, resizes it to 128×128, expands it into a batch, and obtains the model prediction. A threshold of 0.5 is used to determine the displayed class.

5.3 Data Flow

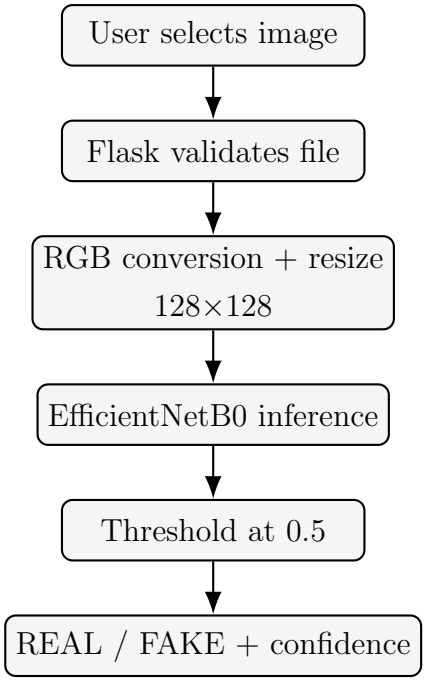


Figure 5.2: Inference data flow

CHAPTER 6

Implementation

6.1 Key Features Implemented

The major implemented features are listed below.

6.1.1 Dataset Loading

The training script uses the project-level directories `train/REAL`, `train/FAKE`, `test/REAL`, and `test/FAKE`. The implementation was adjusted to match the actual extracted dataset structure rather than assuming an additional `dataset/` wrapper directory.

6.1.2 Image Preprocessing

All images are resized to 128×128 pixels. A batch size of 32 is used during training. EfficientNetB0 handles its standard input rescaling internally, consistent with the Keras implementation [10].

6.1.3 Model Construction

The core architecture is EfficientNetB0 with ImageNet-pretrained weights and a binary classification output. The architecture is selected as a practical balance between feature-learning capability and inference cost. The original EfficientNet paper provides the architectural motivation for compound scaling and efficiency [2].

6.1.4 Model Training

The training source contained 100,000 images belonging to two classes. The implemented validation split used 90,000 images for training and 10,000 for validation. A maximum of five epochs was configured. The best-performing model checkpoint was saved as `models/best_model.keras`.

6.1.5 Model Metadata

A JSON metadata file named `model_metadata.json` is saved with the trained model. It records model-related configuration needed by the application and project documentation.

6.1.6 Flask Backend

The Flask backend is responsible for:

- Loading the trained Keras model.
- Accepting image uploads through the web form.
- Reading and converting uploaded images.
- Resizing images to the model input size.
- Running model inference.
- Converting the probability into a REAL/FAKE label.
- Returning the uploaded image and prediction to the frontend.

6.1.7 Frontend

The frontend contains a dark-themed card layout with an image-selection area, image preview, analysis button, and prediction panel. The interface was intentionally kept simple so that the model's output is visible without unnecessary controls.

6.1.8 Inference Logic

The conceptual inference procedure is:

1. Receive an uploaded image.
2. Convert it to RGB.
3. Resize it to 128×128.
4. Convert it into a batch of one image.
5. Pass the image through the saved EfficientNetB0 classifier.
6. Read the scalar probability.
7. If the probability is at least 0.5, display FAKE; otherwise display REAL.

8. Display the corresponding confidence value.

6.1.9 Project Directory Structure

```
CIFAKE_FINAL_VSCODE_PY313/
|-- app.py
|-- train_model.py
|-- requirements.txt
|-- README.md
|-- models/
|   |-- best_model.keras
|   `-- model_metadata.json
|-- templates/
|   `-- index.html
|-- static/
|   |-- style.css
|   `-- uploads/
|-- train/
|   |-- REAL/
|   `-- FAKE/
`-- test/
    |-- REAL/
    `-- FAKE/
```

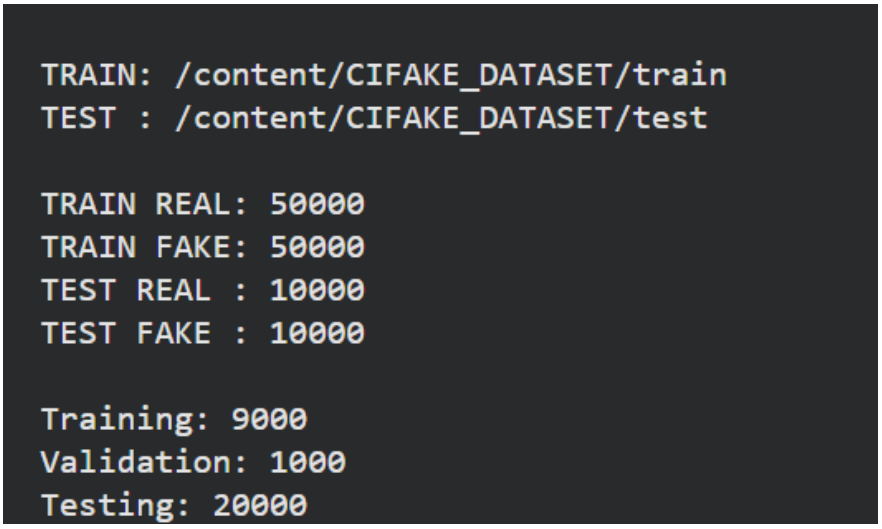


Figure 6.1: CIFAKE dataset structure and dataset distribution

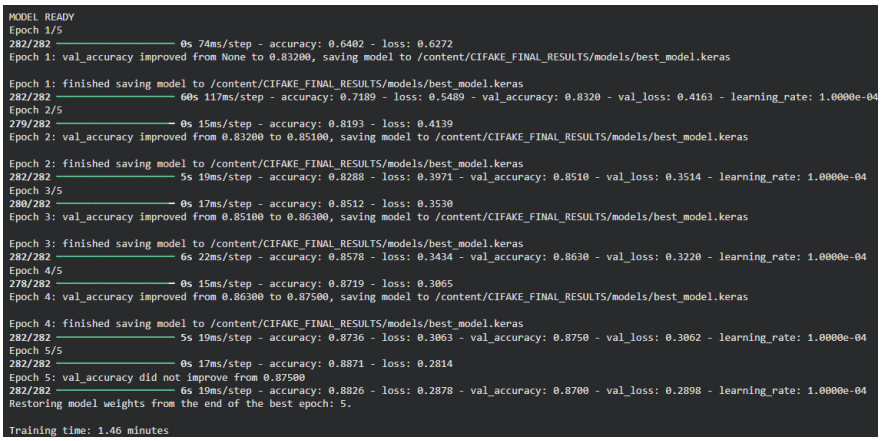


Figure 6.2: EfficientNetB0 model training over five epochs

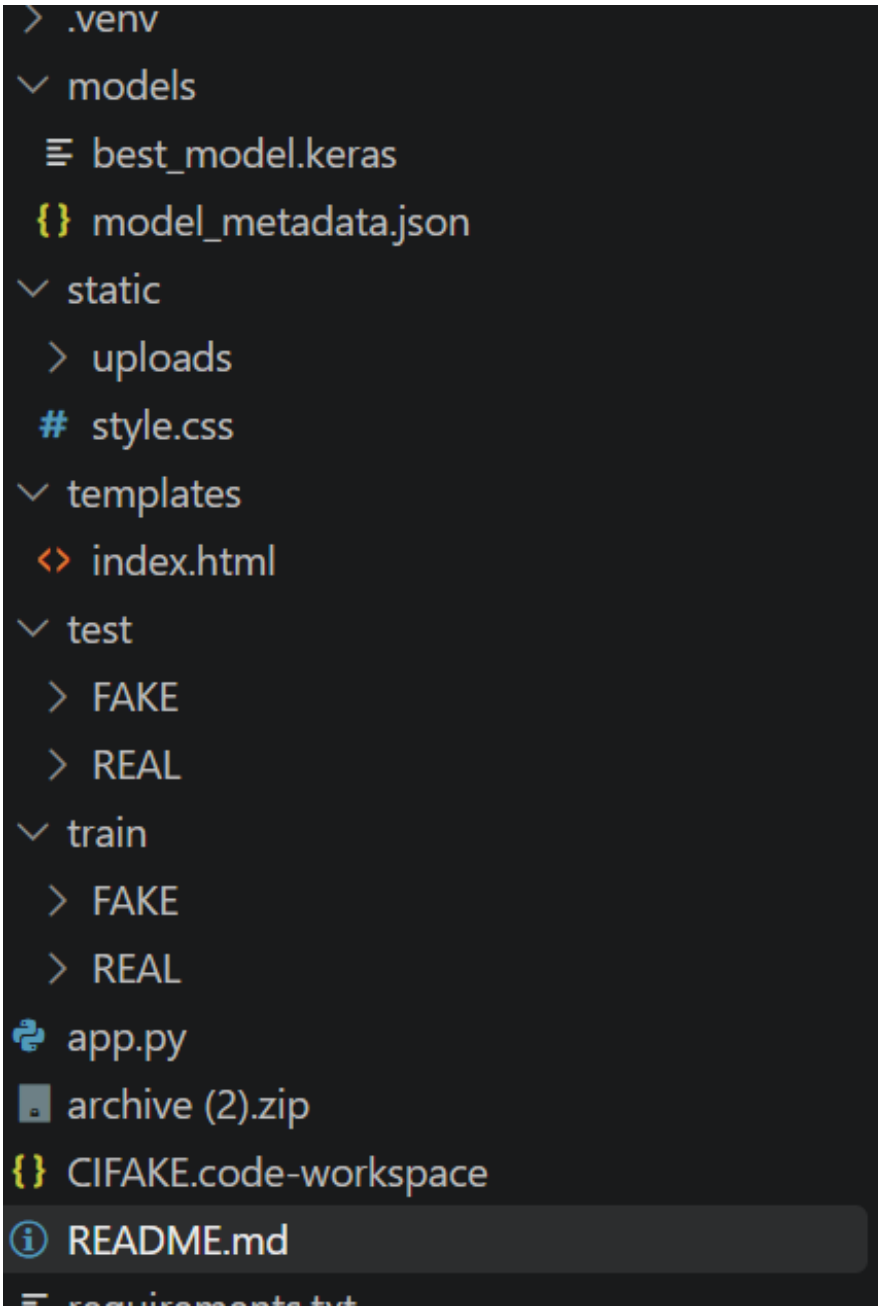


Figure 6.3: Project structure of the CIFAKE detection system

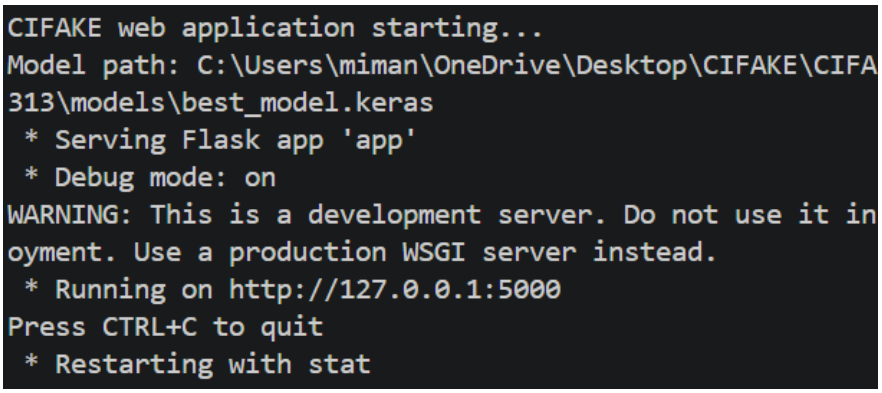


Figure 6.4: Flask backend running successfully in the VS Code terminal

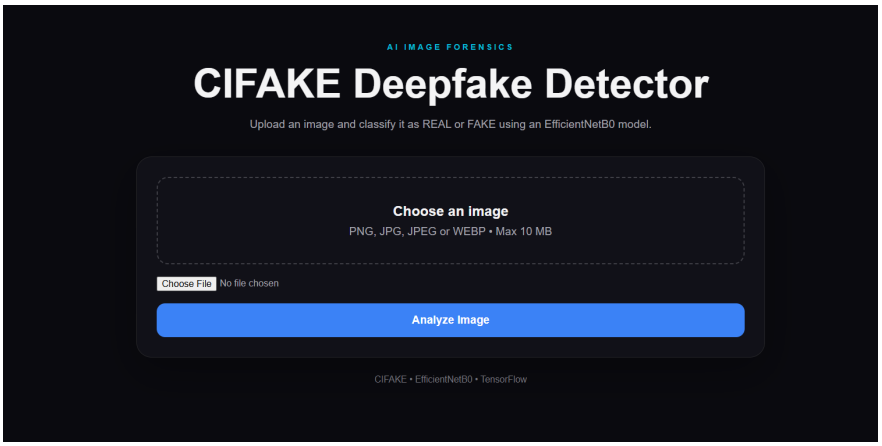


Figure 6.5: Web interface of the CIFAKE Deepfake Image Detector

CHAPTER 7

Results and Discussion

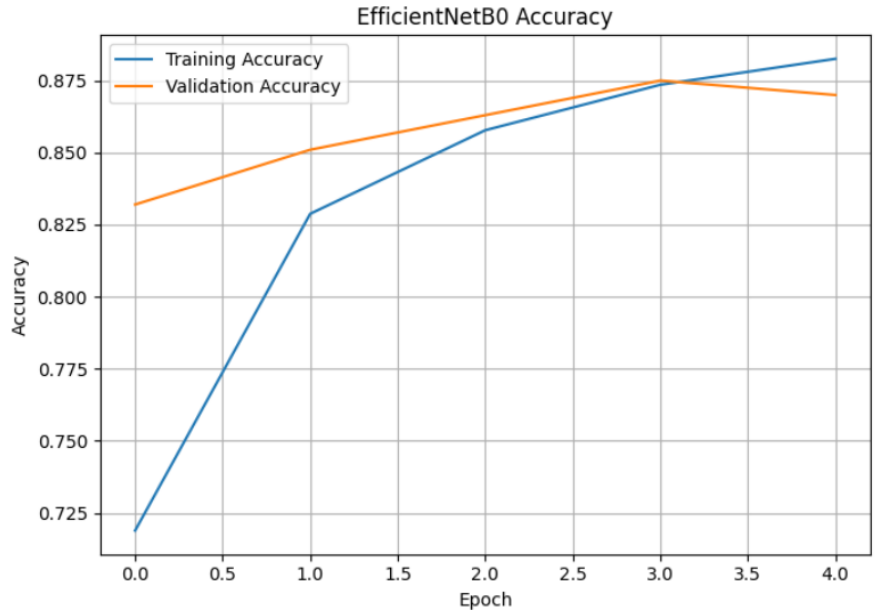


Figure 7.1: Training and validation accuracy of the EfficientNetB0 model

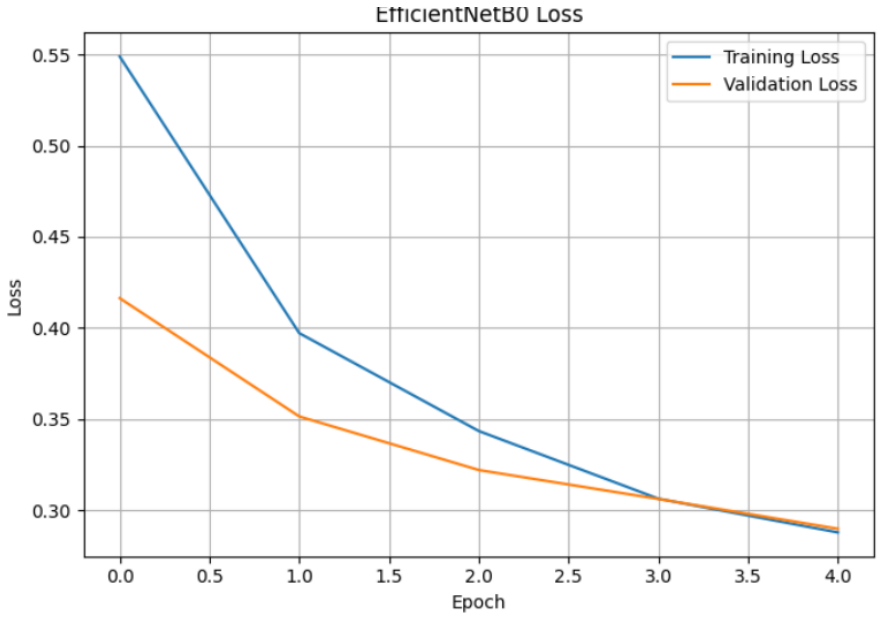


Figure 7.2: Training and validation loss of the EfficientNetB0 model

```
================================================
CIFAKE FINAL TEST RESULTS
================================================
Accuracy : 0.8774
Precision: 0.8876
Recall   : 0.8643
F1 Score : 0.8758
ROC-AUC  : 0.9496

Classification Report:
              precision    recall  f1-score   support

        FAKE       0.87      0.89      0.88     10000
        REAL       0.89      0.86      0.88     10000

    accuracy                           0.88     20000
   macro avg       0.88      0.88      0.88     20000
weighted avg       0.88      0.88      0.88     20000
```

Figure 7.3: Final test-set performance and classification report

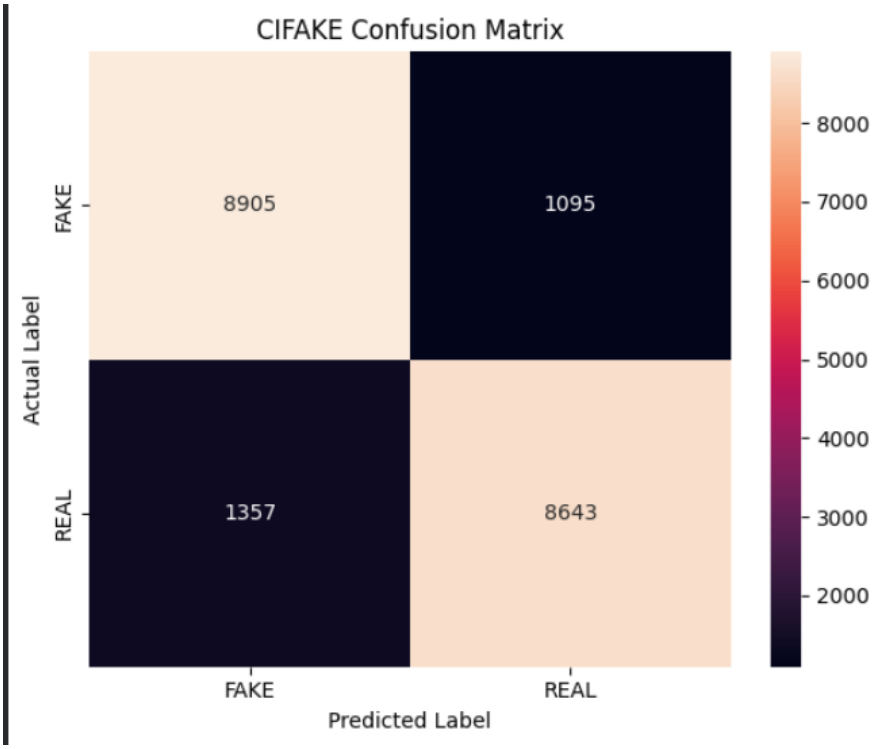


Figure 7.4: Confusion matrix of the CIFAKE classification model

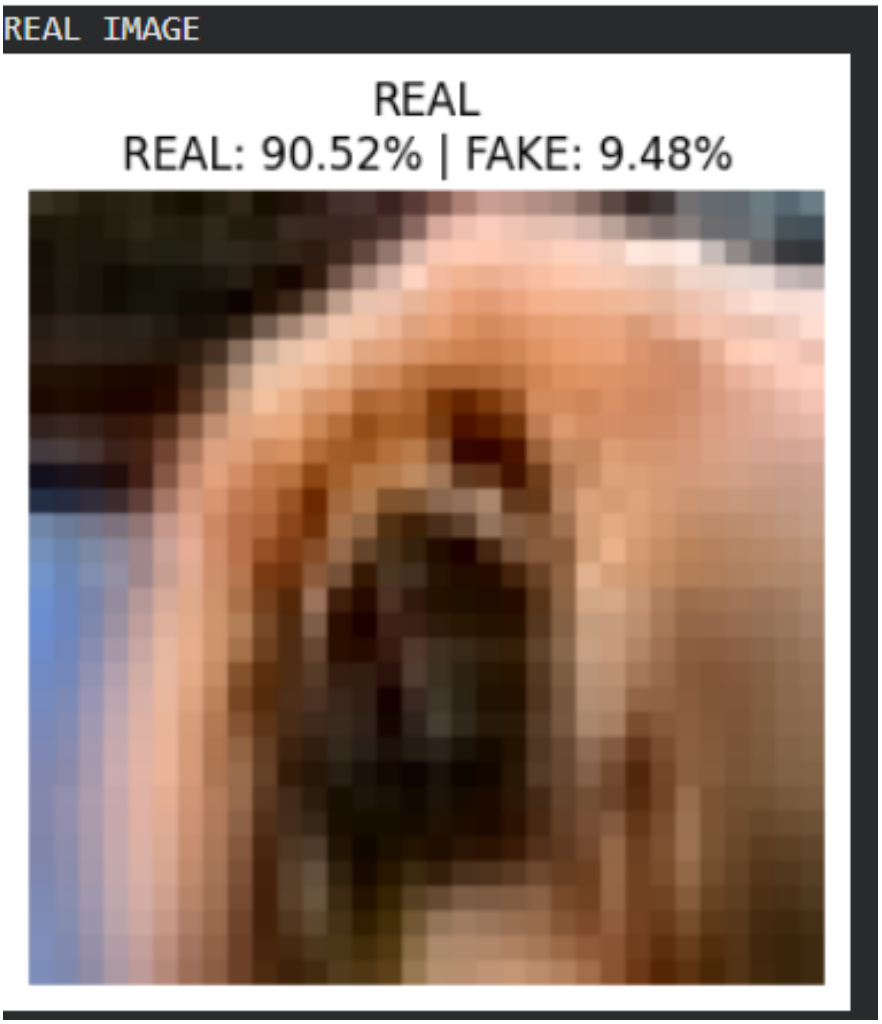


Figure 7.5: Example prediction for a REAL image



Figure 7.6: Example prediction for a FAKE image

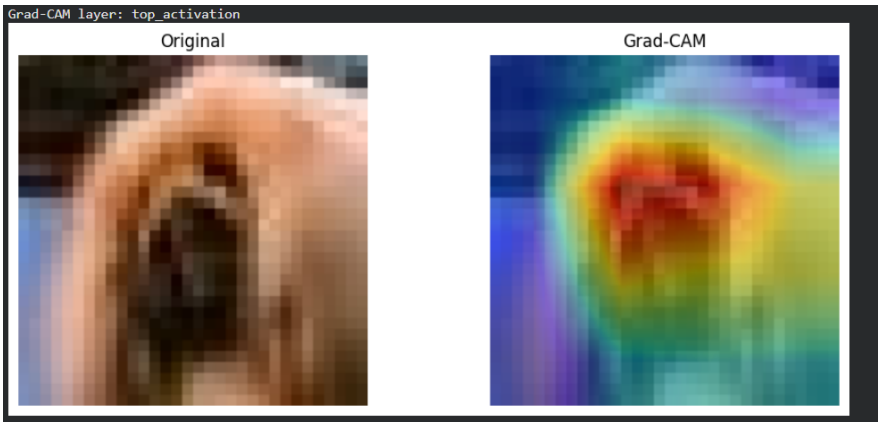


Figure 7.7: Grad-CAM visualization for model interpretability

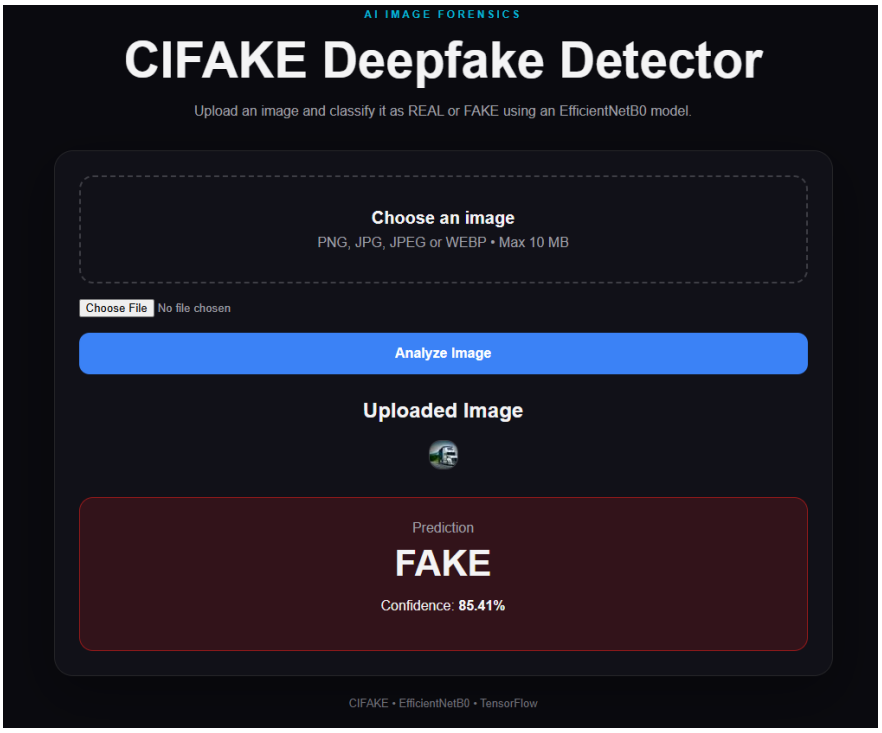


Figure 7.8: Detection result for a FAKE image

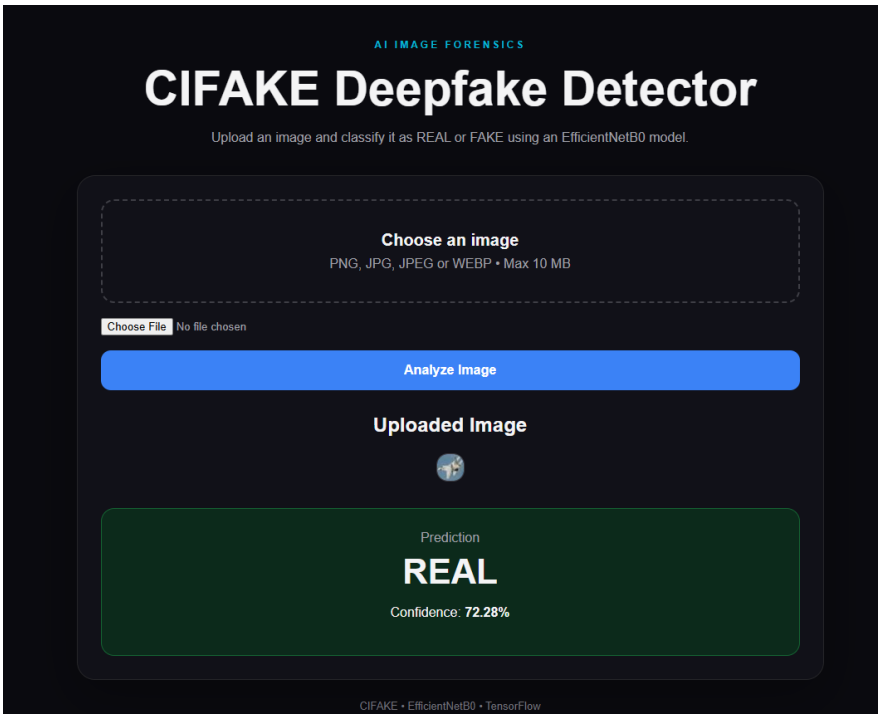


Figure 7.9: Detection result for a REAL image

7.1 Output Analysis

The completed implementation successfully progressed through the complete pipeline: dataset discovery, model training, model saving, Flask startup, image upload, and prediction. The application generated successful HTTP responses for the upload and static-file requests during testing, confirming that the browser frontend and Flask backend were communicating correctly.

The model artifact `best_model.keras` was successfully created with a file size of approximately 17 MB, and `model_metadata.json` was also generated. This confirms that the trained model can be separated from the training process and reused by the web application.

During practical testing, images from the CIFAKE training and test folders were uploaded through the interface for both REAL and FAKE categories. Additional external images were also tested. External images should be interpreted cautiously because their generation process, visual domain, and post-processing may differ from the CIFAKE training distribution.

7.2 Performance Evaluation

The recorded evaluation output from the completed training run was:

Table 7.1: Recorded model performance from the completed run

Metric	Recorded value
Accuracy	88.68%
Precision	88.00%
Recall	89.57%
Loss	0.2744

Accuracy measures the proportion of correctly classified examples. Precision indicates the proportion of predicted positive examples that are correct, while recall indicates the proportion of actual positive examples detected by the model. In this binary setup, the positive class corresponds to the FAKE class as configured by the project.

The reported metrics provide evidence that the trained model learned useful discriminative features from the project data. However, these values should not be interpreted as a universal measure of deepfake-detection reliability. Performance can change when

the source generator, image quality, compression, resolution, or manipulation method changes. This limitation is consistent with current research and evaluation work emphasizing robustness and generalization [7, 6].

7.3 Testing Results

Table 7.2: Functional testing of the deployed application

ID	Test case	Expected behavior	Status
T1	Open local Flask application	Home page loads successfully	Passed
T2	Upload REAL image from test directory	Image is accepted and prediction is displayed	Passed
T3	Upload FAKE image from test directory	Image is accepted and prediction is displayed	Passed
T4	Upload image from training REAL directory	Image is processed by the application	Passed
T5	Upload image from training FAKE directory	Image is processed by the application	Passed
T6	Display uploaded image	Uploaded image appears on result page	Passed
T7	Model loading	Saved Keras model is loaded by Flask	Passed
T8	External image test	Application accepts a new image and returns a model prediction	Passed

7.4 Discussion

The project demonstrates that a deep-learning image classifier can be packaged as a usable local application rather than remaining only as a notebook experiment. The separation between training and inference is an important implementation feature: the expensive training process is completed once, while the Flask application loads the saved model and performs inference on demand.

The testing also illustrates an important distinction between *benchmark performance* and *deployment performance*. A model can perform well on images drawn from a dataset

similar to its training distribution and still make incorrect predictions on external images. Therefore, a production-grade system would require cross-dataset testing, generator-diverse benchmarks, calibration, robustness analysis, and possibly human review. NIST's current evaluation work similarly emphasizes systematic testing of image discriminators and robustness to changing generation and post-processing conditions [6, 7].

CHAPTER 8

Challenges and Learning Outcomes

8.1 Challenges Faced

1. **Dataset path mismatch:** The extracted dataset contained `train/` and `test/` directly at the project root, while the initial code expected `dataset/train/` and `dataset/test/`.
2. **Python and TensorFlow compatibility:** The environment initially used dependency versions that were not appropriate for the selected Python version. The final working environment used Python 3.13.15 with TensorFlow 2.20.0.
3. **Dataset class mapping error:** After TensorFlow's dataset was prefetched, the training script attempted to access `class_names` from the prefetched dataset object. The issue was resolved by using the known mapping `REAL=0, FAKE=1` in the project code.
4. **Training time:** Training on 90,000 images on CPU required substantial computation time, making efficient model selection and reuse important.
5. **Generalization:** Some external images may differ significantly from the CIFAKE distribution. This makes external testing more difficult than testing images sampled from the same benchmark.

8.2 Solutions Implemented

The dataset paths were corrected to point directly to the root-level `train/` and `test/` directories. The TensorFlow environment was rebuilt using compatible package versions. The class-mapping print statement was corrected after dataset prefetching. The trained model was saved so that the web application did not require retraining. Finally, the project was tested with both benchmark images and external images to observe the difference between in-distribution and out-of-distribution behavior.

8.3 Technical Skills Learned

The project provided practical experience in:

- Dataset organization and supervised image classification.
- TensorFlow/Keras model development.
- Transfer learning and pretrained CNN architectures.
- EfficientNetB0-based image classification.
- Training, validation, and evaluation workflows.
- Model checkpointing and serialized model deployment.
- Python virtual environments and dependency management.
- Flask backend development.
- HTML/CSS frontend development.
- Local web deployment and browser-based model inference.
- Debugging Python, TensorFlow, file-path, and deployment errors.
- Interpreting accuracy, precision, recall, and loss.
- Understanding the difference between benchmark testing and external generalization.

CHAPTER 9

Future Enhancements

9.1 Proposed Improvements

Future work can improve the project in several directions. First, the training data can be expanded with images generated by multiple contemporary image-generation systems. This would reduce dependence on a single synthetic-image distribution. Second, cross-dataset evaluation can be added so that a model trained on one source is tested on another source. Third, robustness experiments can be performed after resizing, compression, blur, noise addition, and other transformations.

9.2 Additional Features

Potential additional features include:

- Grad-CAM or related explainable-AI visualizations to highlight image regions influencing the decision.
- Probability calibration and clearer uncertainty reporting.
- Batch image analysis.
- Drag-and-drop upload and improved frontend accessibility.
- Detailed prediction history for a local session.
- Video-frame analysis for synthetic-video detection.
- Multi-model comparison between EfficientNet, ConvNeXt, Vision Transformer, and other architectures.
- Forensic indicators such as compression analysis and metadata inspection as supplementary evidence.

9.3 Scalability Opportunities

The current implementation is a local Flask prototype. It can be scaled by separating the frontend, API, model-serving layer, and storage layer. A production deployment

could use a dedicated inference service, containerization, request queues, monitoring, authentication, and cloud object storage. Batch inference and hardware acceleration could also improve throughput.

For a responsible production system, scalability should be accompanied by stronger evaluation. NIST's current forensic work emphasizes the need to understand robustness, post-processing effects, and performance when models encounter newer generation methods [7]. Therefore, scaling the application should not be treated as a substitute for improving detector generalization.

CHAPTER 10

Conclusion

10.1 Summary of Work Completed

The project developed a complete CIFAKE-based image-classification application for distinguishing REAL and FAKE images. The workflow began with dataset preparation and directory verification, followed by image preprocessing and EfficientNetB0-based model training. The trained model was saved as `best_model.keras` together with metadata. A Flask application was then developed to load the saved model, accept image uploads, preprocess the uploaded image, perform inference, and display the prediction through a browser interface.

The final project was tested using images from both the training and test portions of the CIFAKE dataset, covering REAL and FAKE classes. Additional external images were also used to demonstrate that the deployed application can process new inputs, while also showing the importance of recognizing distribution differences when interpreting individual predictions.

10.2 Achievements

The major achievements of the project are:

- Successful setup of a working TensorFlow/Keras development environment.
- Successful loading of 100,000 training-source images across two classes and 20,000 test images.
- Successful training of an EfficientNetB0-based binary classifier.
- Recorded evaluation output of 88.68% accuracy, 88.00% precision, 89.57% recall, and 0.2744 loss.
- Successful creation of the reusable `best_model.keras` model file.
- Successful integration of the trained model with Flask.
- Successful browser-based image upload and prediction.

- Successful testing with REAL and FAKE dataset images and additional external images.

10.3 Project Impact

The project provides an accessible demonstration of how synthetic-image detection research can be transformed into an interactive application. It also demonstrates an important practical lesson: model accuracy on a benchmark is only one part of a reliable detection system. Real-world deployment requires careful consideration of dataset diversity, unseen generators, post-processing, uncertainty, and human oversight.

10.4 Final Remark

The CIFAKE Deepfake Image Detector establishes a functional foundation for AI-generated image classification. The combination of a benchmark dataset, EfficientNetB0 model, saved inference artifact, and Flask interface provides a complete internship implementation from data to deployment. Future improvements should focus on broader data, stronger cross-generator testing, explainability, robustness, and responsible use rather than treating a single classification output as definitive proof of image authenticity.

Bibliography

[1] J. J. Bird and A. Lotfi, "CIFAKE: Image Classification and Explainable Identification of AI-Generated Synthetic Images," arXiv preprint arXiv:2303.14126, 2023. Available: https://arxiv.org/abs/2303.14126.

[2] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019. Available: https://arxiv.org/abs/1905.11946.

[3] T. T. Nguyen, Q. V. H. Nguyen, D. T. Nguyen, D. T. Nguyen, T. Huynh-The, S. Nahavandi, T. T. Nguyen, Q.-V. Pham, C. M. Nguyen, "Deep Learning for Deepfakes Creation and Detection: A Survey," arXiv preprint arXiv:1909.11573, 2019. Available: https://arxiv.org/abs/1909.11573.

[4] L. A. Passos, D. Jodas, K. A. P. da Costa, L. A. Souza Júnior, D. Rodrigues, J. Del Ser, D. Camacho, and J. P. Papa, "A Review of Deep Learning-based Approaches for Deepfake Content Detection," arXiv preprint arXiv:2202.06095, 2022. Available: https://arxiv.org/abs/2202.06095.

[5] National Institute of Standards and Technology, "Synthetic Image Detection Techniques," NIST AI 100-4, 2024. Available: https://airc.nist.gov/docs/NIST.AI.100-4.SyntheticContent.ipd.pdf.

[6] G. Awad, H. Iyer, S. Seo, P. Fontana, and Y. Lee, "2025 NIST GenAI (Pilot) Evaluation Plan for Image Discriminators," National Institute of Standards and Technology, 2025. Available: https://www.nist.gov/publications/2025-nist-genai-pilot-evaluation-plan-image-discriminators.

[7] H. Guan, J. Horan, and A. Zhang, "Guardians of Forensic Evidence: Evaluating Analytic Systems Against AI-Generated Deepfakes," NIST Forensics@NIST 2024, 2025. Available: NIST publication page.

[8] H. Guan, Y. Lee, and L. Diduch, "Open Media Forensics Challenge 2022 Evaluation Plan," National Institute of Standards and Technology, 2022. Available: NIST evaluation plan.

[9] TensorFlow, "Install TensorFlow with pip," TensorFlow Documentation. Available: https://www.tensorflow.org/install/pip.

[10] TensorFlow, "tf.keras.applications.EfficientNetB0," TensorFlow API Documentation. Available: https://www.tensorflow.org/api_docs/python/tf/keras/applications/EfficientNetB0.